

Documentation du Module de Gestion des Agences

Vue d'ensemble

Ce document explique tout le module `agence` de l'API de gestion immobiliere. L'objectif est de decrire clairement:

- 1. Les endpoints associes a `agence` .
- 2. Les donnees attendues (body, params).
- 3. Les fonctions appelees dans l'ordre.
- 4. Le role de chaque couche (route, validation, controller, service, repository, reponse, erreurs).

Le module `agence` expose les endpoints suivants:

- 1. `GET /api/agences`
- 2. `GET /api/agences/:id`
- 3. `POST /api/agences`
- 4. `PUT /api/agences/:id`
- 5. `DELETE /api/agences/:id`

Architecture en couches:

- 1. Route
- 2. Validation
- 3. Controller
- 4. Services
- 5. Repositories
- 6. Reponse standard et gestion d'erreurs

Structure des donnees

1) Body de creation (`POST /api/agences`)

```
{
  "code": "DKR-02",
  "nom": "Agence Dakar Nord",
  "adresse": "Yoff, Dakar"
}
```

Regles principales:

- 1. `code` : obligatoire, format `AA-00` ou `AAA-00` (ex: `DKR-01`).
- 2. `nom` : texte obligatoire, 2 a 100 caracteres.
- 3. `adresse` : texte obligatoire, 5 a 255 caracteres.
- 4. `code` doit etre unique en base.

2) Body de mise a jour (`PUT /api/agences/:id`)

```
{
  "nom": "Agence Dakar Centre",
  "adresse": "Plateau, Dakar"
}
```

Regles principales:

- 1. Au moins un champ requis.
- 2. Schema actuel: `nom` et `adresse` .
- 3. `id` dans l'URL doit etre un entier valide.

3) Parametre path (`:id`)

```
/api/agences/:id
```

- 1. L'ID est converti avec `convertirId(...)` .
- 2. Si l'ID est invalide, erreur `400` .

4) Modele Prisma cible

```
model Agence {
  id      Int      @id @default(autoincrement())
  code    String   @unique
  nom     String
  adresse String
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  clients Client[]
  biens    Bien[]

  @@map("agences")
}
```

Documentation des endpoints

1. Endpoint GET /api/agences

```
router.get('/', agenceController.getAgences);
```

Role principal: recuperer toutes les agences.

Fonctionnement:

1. Appelle `agenceService.trouverToutesAgences()`.
2. Retourne la liste triee par nom.
3. Renvoie HTTP `200`.

2. Endpoint GET /api/agences/:id

```
router.get('/:id', agenceController.getAgenceById);
```

Role principal: recuperer une agence par son ID.

Fonctionnement:

1. Lit `id` depuis `req.params`.
2. Appelle `agenceService.trouverAgenceParId(id)`.
3. Renvoie `200` si trouve, sinon `404`.

3. Endpoint POST /api/agences

```
router.post('/', valider(schemaCreerAgence), agenceController.createAgence);
```

Role principal: creer une nouvelle agence.

Fonctionnement:

1. Valide le body avec `schemaCreerAgence`.
2. Verifie unicite du `code`.
3. Cree l'agence.
4. Renvoie HTTP `201`.

4. Endpoint PUT /api/agences/:id

```
router.put('/:id', valider(schemaMettreAJourAgence), agenceController.updateAgence);
```

Role principal: mettre a jour une agence existante.

Fonctionnement:

- 1. Valide le body de mise a jour.
- 2. Verifie que l'agence existe.
- 3. Met a jour les champs autorises.
- 4. Renvoie HTTP 200 .

5. Endpoint DELETE /api/agences/:id

```
router.delete('/:id', agenceController.deleteAgence);
```

Role principal: supprimer une agence si aucune relation bloquante.

Fonctionnement:

- 1. Verifie que l'agence existe.
- 2. Compte les clients et biens lies.
- 3. Bloque la suppression si relations existantes (409).
- 4. Supprime et renvoie 200 sinon.

Documentation des fonctions

1. schemaCreerAgence

```
const schemaCreerAgence = Joi.object({
  code: Joi.string().pattern(/^[A-Z]{2,3}-\d{2}$/).required(),
  nom: Joi.string().min(2).max(100).required(),
  adresse: Joi.string().min(5).max(255).required(),
});
```

Role principal: valider le body de creation.

Fonctionnement:

- 1. Controle format du code .
- 2. Controle nom et adresse .
- 3. Rejette la requete si invalide.

2. `schemaMettreAJourAgence`

```
const schemaMettreAJourAgence = Joi.object({
  nom: Joi.string().min(2).max(100),
  adresse: Joi.string().min(5).max(255),
}).min(1);
```

Role principal: valider le body de mise a jour.

Fonctionnement:

1. Rend les champs optionnels.
2. Exige au moins un champ.
3. Nettoie les champs non prevus via middleware `valider`.

3. `valider(schema, source = 'body')`

```
const { error, value } = schema.validate(donnees, {
  abortEarly: false,
  stripUnknown: true,
});
```

Role principal: middleware generique de validation.

Fonctionnement:

1. Valide les donnees entrantes.
2. Nettoie les champs non autorises.
3. Passe une erreur `400` si echec.

4. `getAgences()`

```
const getAgences = asyncHandler(async (req, res) => {
  const agences = await agenceService.trouverToutesAgences();
  sendResponse(res, 200, 'Agences récupérées avec succès', agences);
});
```

Role principal: controller de lecture liste.

Fonctionnement:

1. Interroge le service.
2. Retourne toutes les agences avec `200`.

5. `getAgenceById()`

```
const getAgenceById = asyncHandler(async (req, res) => {  
  const { id } = req.params;  
  const agence = await agenceService.trouverAgenceParId(id);  
  sendResponse(res, 200, 'Agence récupérée avec succès', agence);  
});
```

Role principal: controller de lecture detail.

Fonctionnement:

1. Recupere `id` depuis l'URL.
2. Delege au service.
3. Retourne `404` si agence absente.

6. `createAgence()`

```
const createAgence = asyncHandler(async (req, res) => {  
  const { code, nom, adresse } = req.body;  
  const existant = await agenceService.trouverAgenceParCode(code);  
  if (existant) throw new HttpError(409, `Une agence avec le code ${code} existe déjà`);  
  
  const agence = await agenceService.creerAgence({ code, nom, adresse });  
  sendResponse(res, 201, 'Agence créée avec succès', agence);  
});
```

Role principal: controller de creation.

Fonctionnement:

1. Lit les donnees valides.
2. Controle doublon sur `code`.
3. Cree l'agence et renvoie `201`.

7. `updateAgence()`

```
const updateAgence = asyncHandler(async (req, res) => {  
  const { id } = req.params;  
  const { nom, adresse, code } = req.body;  
  const agence = await agenceService.mettreAJourAgence(id, { code, nom, adresse });  
  sendResponse(res, 200, 'Agence mise à jour avec succès', agence);  
});
```

Role principal: controller de mise a jour.

Fonctionnement:

1. Verifie l'existence de l'agence.
2. Applique la mise a jour.
3. Retourne `200`.

8. `deleteAgence()`

```
const deleteAgence = asyncHandler(async (req, res) => {  
  const { id } = req.params;  
  await agenceService.supprimerAgence(id);  
  sendResponse(res, 200, 'Agence supprimée avec succès');  
});
```

Role principal: controller de suppression.

Fonctionnement:

1. Verifie l'existence de l'agence.
2. Verifie l'absence de relations bloquantes.
3. Supprime et renvoie `200`.

9. `trouverToutesAgences()` dans `agence.service`

```
async function trouverToutesAgences() {  
  return agenceRepo.trouverToutesAgences();  
}
```

Role principal: deleguer la lecture liste au repository.

10. `trouverAgenceParId(id)` dans `agence.service`

```
async function trouverAgenceParId(id) {  
  const agence = await agenceRepo.trouverAgenceParId(id);  
  if (!agence) throw new HttpError(404, 'Agence non trouvée');  
  return agence;  
}
```

Role principal: regle metier d'existence agence.

11. `creerAgence(donnees)` dans `agence.service`

```
async function creerAgence(donnees) {  
  const existant = await agenceRepo.trouverAgenceParCode(donnees.code);  
  if (existant) throw new HttpError(409, `Une agence avec le code ${donnees.code} existe déjà`);  
  return agenceRepo.creer(donnees);  
}
```

Role principal: logique metier de creation agence.

12. `supprimerAgence(id)` dans `agence.service`

```
async function supprimerAgence(id) {  
  await trouverAgenceParId(id);  
  const { aDesRelations, clients, biens } = await agenceRepo.agenceADesRelations(id);  
  if (aDesRelations) {  
    throw new HttpError(409, `Impossible de supprimer : l'agence a ${clients} client(s) et ${biens} bien(s) liés`);  
  }  
  return agenceRepo.supprimer(id);  
}
```

Role principal: logique metier de suppression.

13. `trouverToutesAgences()` dans `agence.repo`

```
async function trouverToutesAgences() {
  return prisma.agence.findMany({
    orderBy: { nom: 'asc' },
    include: { _count: { select: { clients: true, biens: true } } },
  });
}
```

Role principal: lire toutes les agences avec compteurs.

14. `trouverAgenceParId(id)` dans `agence.repo`

```
async function trouverAgenceParId(id) {
  return agenceBase.trouverParId(id, {
    include: {
      clients: { select: { id: true, prenom: true, nom: true, email: true } },
      biens: { select: { id: true, titre: true, statut: true, prix: true } },
      _count: { select: { clients: true, biens: true } },
    },
  });
}
```

Role principal: lire une agence avec ses relations.

15. `agenceADesRelations(id)` dans `agence.repo`

```
async function agenceADesRelations(id) {
  const agenceId = convertirId(id, "ID de l'agence");
  const [clients, biens] = await Promise.all([
    prisma.client.count({ where: { agenceId } }),
    prisma.bien.count({ where: { agenceId } }),
  ]);
  return { aDesRelations: clients > 0 || biens > 0, clients, biens };
}
```

Role principal: verifier les contraintes de suppression.

16. `sendResponse(...)`, `asyncHandler(...)`, `errorHandler(...)`

Role principal: standardiser reponses et erreurs.

Fonctionnement:

- 1. `sendResponse` renvoie un JSON uniforme.
- 2. `asyncHandler` capture les erreurs async.
- 3. `errorHandler` convertit erreurs metier/Prisma en HTTP.

Resume du flux complet du module `agence`

- 1. La route choisit le controller selon l'endpoint.
- 2. `POST` et `PUT` passent par la validation Joi.
- 3. Le controller orchestre les verifications.
- 4. Le service applique les regles metier.
- 5. Le repository execute les requetes Prisma.
- 6. `sendResponse` renvoie un format JSON standard.
- 7. `errorHandler` formate les erreurs en reponses HTTP coherentes.

Documentation du Module de Gestion des Clients

Vue d'ensemble

Ce document explique tout le module `client` de l'API de gestion immobiliere. L'objectif est de decrire clairement:

- 1. Les endpoints associes a `client` .
- 2. Les donnees attendues (body, query, params).
- 3. Les fonctions appelees dans l'ordre.
- 4. Le role de chaque couche (route, validation, controller, service, repository, reponse, erreurs).

Le module `client` expose les endpoints suivants:

- 1. `GET /api/clients`
- 2. `GET /api/clients/:id`
- 3. `POST /api/clients`
- 4. `PUT /api/clients/:id`
- 5. `DELETE /api/clients/:id`

Architecture en couches:

- 1. Route
 - 2. Validation
 - 3. Controller
 - 4. Services
 - 5. Repositories
 - 6. Reponse standard et gestion d'erreurs
-

Structure des donnees

1) Body de creation (`POST /api/clients`)

```
{
  "prenom": "Awa",
  "nom": "Diop",
  "email": "awa.diop@email.com",
  "telephone": "+221771112233",
  "agenceId": 1
}
```

Regles principales:

1. `prenom` : obligatoire, 2 a 50 caracteres.
2. `nom` : obligatoire, 2 a 50 caracteres.
3. `email` : obligatoire et format email valide.
4. `telephone` : optionnel, format numero valide.
5. `agenceId` : entier obligatoire, reference une agence existante.
6. `email` et `telephone` doivent etre uniques.

2) Body de mise a jour (`PUT /api/clients/:id`)

```
{
  "prenom": "Awa",
  "nom": "Ndiaye",
  "telephone": "+221781112233"
}
```

Regles principales:

1. Tous les champs sont optionnels.
2. Au moins un champ doit etre present.
3. Si `email` est modifie, l'unicite est verifiee.
4. Si `agenceId` est fourni, l'agence doit exister.

3) Query params (`GET /api/clients`)

```
agenceId, recherche
```

Notes:

1. `agenceId` : filtre par agence.
2. `recherche` : recherche texte sur `nom`, `prenom`, `email`.

4) Parametre path (`:id`)

```
/api/clients/:id
```

1. L'ID est converti avec `convertirId(...)`.
2. Si invalide, erreur `400`.

5) Modele Prisma cible

```
model Client {
  id          Int      @id @default(autoincrement())
  prenom      String
  nom         String
  email       String   @unique
  telephone   String   @unique
  agenceId    Int
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  agence      Agence   @relation(fields: [agenceId], references: [id])
  visites     Visite[]

  @@map("clients")
}
```

Documentation des endpoints

1. Endpoint `GET /api/clients`

```
router.get('/', clientController.getClients);
```

Role principal: recuperer la liste des clients avec filtres optionnels.

Fonctionnement:

1. Lit `agenceId` et `recherche` dans la query.
2. Appelle `clientService.trouverTousClients(filtres)`.

3. Renvoie HTTP `200`.

2. Endpoint `GET /api/clients/:id`

```
router.get('/:id', clientController.getClientById);
```

Role principal: recuperer un client par son ID.

Fonctionnement:

1. Lit `id` depuis l'URL.
2. Appelle `clientService.trouverClientParId(id)`.
3. Renvoie `200` si trouve, sinon `404`.

3. Endpoint `POST /api/clients`

```
router.post('/', valider(schemaCreerClient), clientController.createClient);
```

Role principal: creer un nouveau client.

Fonctionnement:

1. Valide le body avec `schemaCreerClient`.
2. Controle unicite email/telephone.
3. Verifie l'existence de l'agence.
4. Cree le client et renvoie `201`.

4. Endpoint `PUT /api/clients/:id`

```
router.put('/:id', valider(schemaMettreAJourClient), clientController.updateClient);
```

Role principal: mettre a jour un client existant.

Fonctionnement:

1. Valide le body.
2. Verifie l'existence du client.
3. Controle l'unicite email (si modifie).
4. Verifie l'agence (si `agenceId` fourni).
5. Renvoie `200`.

5. Endpoint `DELETE /api/clients/:id`

```
router.delete('/:id', clientController.deleteClient);
```


Role principal: supprimer un client si aucune visite liee.

Fonctionnement:

- 1. Verifie que le client existe.
- 2. Verifie qu'il n'a pas de visites.
- 3. Supprime et renvoie 200 .
- 4. Renvoie 409 si conflit metier.

Documentation des fonctions

1. schemaCreerClient

```
const schemaCreerClient = Joi.object({
  prenom: Joi.string().min(2).max(50).required(),
  nom: Joi.string().min(2).max(50).required(),
  email: Joi.string().email().required(),
  telephone: Joi.string().regex(/^+221(77|78|76)[0-9]{7}$/),
  agenceId: Joi.number().integer().required(),
});
```

Role principal: valider le body de creation client.

2. schemaMettreAJourClient

```
const schemaMettreAJourClient = Joi.object({
  prenom: Joi.string().min(2).max(50),
  nom: Joi.string().min(2).max(50),
  email: Joi.string().email(),
  telephone: Joi.string().pattern(/^(\+?[0-9]{7,15})$/),
  agenceId: Joi.number().integer(),
}).min(1);
```

Role principal: valider le body de mise a jour.

3. valider(schema, source = 'body')

```
const { error, value } = schema.validate(donnees, {
  abortEarly: false,
  stripUnknown: true,
});
```

Role principal: middleware de validation generique.

4. `getClients()`

```
const getClients = asyncHandler(async (req, res) => {  
  const { agenceId, recherche } = req.query;  
  const filtros = {};  
  if (agenceId) filtros.agenceId = agenceId;  
  if (recherche) filtros.recherche = recherche;  
  
  const clients = await clientService.trouverTousClients(filtros);  
  sendResponse(res, 200, 'Clients récupérés avec succès', clients);  
});
```

Role principal: controller de lecture liste clients.

5. `getClientById()`

```
const getClientById = asyncHandler(async (req, res) => {  
  const { id } = req.params;  
  const client = await clientService.trouverClientParId(id);  
  sendResponse(res, 200, 'Client récupéré avec succès', client);  
});
```

Role principal: controller de lecture detail client.

6. createClient()

```
const createClient = asyncHandler(async (req, res) => {
  const { prenom, nom, email, telephone, agenceId } = req.body;

  const emailExistant = await clientService.trouverClientParEmail(email);
  if (emailExistant) throw new HttpError(409, `Un client avec l'email ${email} existe déjà`);

  if (telephone) {
    const telephoneExistant = await clientService.trouverClientParTelephone(telephone);
    if (telephoneExistant) throw new HttpError(409, `Un client avec le téléphone ${telephone} existe déjà`);
  }

  await agenceService.trouverAgenceParId(agenceId);

  const client = await clientService.creerClient({
    prenom,
    nom,
    email,
    telephone,
    agenceId: parseInt(agenceId)
  });

  sendResponse(res, 201, 'Client créé avec succès', client);
});
```

Role principal: controller de creation client.

7. `updateClient()`

```
const updateClient = asyncHandler(async (req, res) => {
  const { id } = req.params;
  const { prenom, nom, email, telephone, agenceId } = req.body;

  const client = await clientService.mettreAJourClient(id, {
    prenom,
    nom,
    email,
    telephone,
    agenceId: agenceId ? parseInt(agenceId) : undefined
  });

  sendResponse(res, 200, 'Client mis à jour avec succès', client);
});
```

Role principal: controller de mise a jour client.

8. `deleteClient()`

```
const deleteClient = asyncHandler(async (req, res) => {
  const { id } = req.params;
  await clientService.supprimerClient(id);
  sendResponse(res, 200, 'Client supprimé avec succès');
});
```

Role principal: controller de suppression client.

9. `trouverTousClients(filtres = {})` dans `client.service`

```
async function trouverTousClients(filtres = {}) {
  return clientRepo.trouverTousClients(filtres);
}
```

Role principal: deleguer la lecture liste au repository.

10. `trouverClientParId(id)` dans `client.service`

```
async function trouverClientParId(id) {  
  const client = await clientRepo.trouverClientParId(id);  
  if (!client) throw new HttpError(404, 'Client non trouvé');  
  return client;  
}
```

Role principal: regle metier d'existence client.

11. `creerClient(donnees)` dans `client.service`

```
async function creerClient(donnees) {  
  const existant = await clientRepo.trouverClientParEmail(donnees.email);  
  const existantTel = await clientRepo.trouverClientParTelephone(donnees.telephone);  
  if (existant) throw new HttpError(409, `Un client avec l'email ${donnees.email} existe déjà`);  
  if (existantTel) throw new HttpError(409, `Un client avec le téléphone ${donnees.telephone} existe déjà`);  
  
  const agence = await agenceRepo.trouverAgenceParId(donnees.agenceId);  
  if (!agence) throw new HttpError(400, `L'agence avec l'ID ${donnees.agenceId} n'existe pas`);  
  
  return clientRepo.creer(donnees);  
}
```

Role principal: logique metier de creation client.

12. `mettreAJourClient(id, donnees)` dans `client.service`

```
async function mettreAJourClient(id, donnees) {
  await trouverClientParId(id);
  if (donnees.email) {
    const existant = await clientRepo.trouverClientParEmail(donnees.email);
    if (existant && existant.id !== parseInt(id)) {
      throw new HttpError(409, `Un client avec l'email ${donnees.email} existe déjà`);
    }
  }
  if (donnees.agenceId) {
    const agence = await agenceRepo.trouverAgenceParId(donnees.agenceId);
    if (!agence) throw new HttpError(400, `L'agence avec l'ID ${donnees.agenceId} n'existe pas`);
  }
  return clientRepo.mettreAJour(id, donnees);
}
```

Role principal: logique metier de mise a jour client.

13. `supprimerClient(id)` dans `client.service`

```
async function supprimerClient(id) {
  await trouverClientParId(id);
  const aDesVisites = await clientRepo.clientADesVisites(id);
  if (aDesVisites) throw new HttpError(409, 'Impossible de supprimer : le client a des visites planifiées');
  return clientRepo.supprimer(id);
}
```

Role principal: logique metier de suppression client.

14. `trouverTousClients(filtres = {})` dans `client.repo`

```
async function trouverTousClients(filtres = {}) {
  const where = {};
  if (filtres.agenceId) where.agenceId = convertirId(filtres.agenceId, "ID de l'agence");

  if (filtres.recherche) {
    where.OR = [
      { nom: { contains: filtres.recherche, mode: 'insensitive' } },
      { prenom: { contains: filtres.recherche, mode: 'insensitive' } },
      { email: { contains: filtres.recherche, mode: 'insensitive' } },
    ];
  }

  return prisma.client.findMany({ where });
}
```

Role principal: filtrer et lire la collection clients.

15. `trouverClientParId(id)` dans `client.repo`

```
async function trouverClientParId(id) {
  return clientBase.trouverParId(id, {
    include: {
      agence: { select: { id: true, code: true, nom: true } },
      visites: {
        include: { bien: { select: { id: true, titre: true, adresse: true } } },
        orderBy: { dateVisite: 'desc' }
      }
    }
  });
}
```

Role principal: lire un client avec ses relations.

16. `clientADesVisites(id)` dans `client.repo`

```
async function clientADesVisites(id) {
  const clientId = convertirId(id, "ID du client");
  const count = await prisma.visite.count({ where: { clientId } });
  return count > 0;
}
```

Role principal: verifier contrainte de suppression.

17. `sendResponse(...)`, `asyncHandler(...)`, `errorHandler(...)`

Role principal: standardiser reponses et erreurs.

Fonctionnement:

1. `sendResponse` renvoie une reponse JSON uniforme.
2. `asyncHandler` capture les erreurs async.
3. `errorHandler` mappe erreurs metier/Prisma vers HTTP.

Resume du flux complet du module `client`

1. La route choisit le controller selon l'endpoint.
2. `POST` et `PUT` passent par la validation Joi.
3. Le controller orchestre les controles d'unicite et de references.
4. Le service applique les regles metier.
5. Le repository execute les requetes Prisma.
6. `sendResponse` renvoie un format JSON standard.
7. `errorHandler` formate les erreurs en reponses HTTP coherentes.

Documentation du Module de Gestion des Biens

Vue d'ensemble

Ce document explique tout le module `bien` de l'API de gestion immobiliere. L'objectif est de decrire clairement:

- 1. Les endpoints associes a `bien`.
- 2. Les donnees attendues (body, query, params).
- 3. Les fonctions appelees dans l'ordre.
- 4. Le role de chaque couche (route, validation, controller, service, repository, reponse, erreurs).

Le module `bien` expose les endpoints suivants:

- 1. `GET /api/biens/catalogue`
- 2. `GET /api/biens`
- 3. `GET /api/biens/:id`
- 4. `POST /api/biens`
- 5. `PUT /api/biens/:id`
- 6. `DELETE /api/biens/:id`
- 7. `PATCH /api/biens/:id/archiver`

Architecture en couches:

- 1. Route
 - 2. Validation
 - 3. Controller
 - 4. Services
 - 5. Repositories
 - 6. Reponse standard et gestion d'erreurs
-

Structure des donnees

1) Body de creation (`POST /api/biens`)

```
{
  "titre": "Appartement F3 aux Almadies",
  "type": "APPARTEMENT",
  "adresse": "Almadies, Dakar",
  "prix": 250000,
  "statut": "DISPONIBLE",
  "agenceId": 1
}
```

Regles principales:

1. `titre` : texte obligatoire.
2. `type` : valeur obligatoire (`MAISON` , `APPARTEMENT` , `STUDIO` , `TERRAIN` , `VILLA` , `BUREAU` , `COMMERCE`).
3. `adresse` : texte obligatoire.
4. `prix` : nombre `>= 0` .
5. `statut` : optionnel (`DISPONIBLE` par default).
6. `agenceId` : entier obligatoire, reference une agence existante.

2) Body de mise a jour (`PUT /api/biens/:id`)

```
{
  "titre": "Appartement F4 renove",
  "prix": 300000,
  "statut": "RESERVE"
}
```

Regles principales:

1. Tous les champs sont optionnels.
2. Au moins un champ doit etre present.
3. Si `agenceId` est fourni, il doit pointer vers une agence existante.

3) Parametres query (`GET /api/biens` et `GET /api/biens/catalogue`)

```
type, statut, agenceId, prixMin, prixMax
```

Notes:

- 1. catalogue exclut les biens archives.
- 2. agenceId est converti en entier.
- 3. prixMin et prixMax sont convertis en nombres.

4) Parametre path (:id)

```
/api/biens/:id
```

- 1. L'ID est converti avec convertirId(...).
- 2. Si l'ID est invalide, erreur 400.

5) Modele Prisma cible

```
model Bien {
  id          Int          @id @default(autoincrement())
  titre       String
  type        TypeBien
  adresse     String
  prix        Float
  statut      StatutBien @default(DISPONIBLE)
  agenceId    Int
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  agence      Agence @relation(fields: [agenceId], references: [id])
  visites     Visite[]

  @@map("biens")
}
```

Documentation des endpoints

1. Endpoint GET /api/biens/catalogue

```
router.get('/catalogue', bienController.getCatalogue);
```

Role principal: retourner le catalogue public des biens.

Fonctionnement:

- 1. Lit les filtres query (type , statut , prixMin , prixMax).

2. Force `catalogue: true`.
3. Appelle `bienService.trouverTousBiens(filtres)`.
4. Renvoie HTTP `200`.

2. Endpoint `GET /api/biens`

```
router.get('/', bienController.getBien);
```

Role principal: retourner la liste complete des biens avec filtres optionnels.

Fonctionnement:

1. Lit les filtres query (`type`, `statut`, `agenceId`, `prixMin`, `prixMax`).
2. Appelle `bienService.trouverTousBiens(filtres)`.
3. Renvoie HTTP `200`.

3. Endpoint `GET /api/biens/:id`

```
router.get('/:id', bienController.getBienById);
```

Role principal: recuperer un bien par son identifiant.

Fonctionnement:

1. Lit `id` depuis `req.params`.
2. Appelle `bienService.trouverBienParId(id)`.
3. Renvoie HTTP `200` si trouve.
4. Renvoie `404` si non trouve.

4. Endpoint `POST /api/biens`

```
router.post('/', valider(schemaCreerBien), bienController.createBien);
```

Role principal: creer un nouveau bien.

Fonctionnement:

1. Valide le body avec `schemaCreerBien`.
2. Verifie que l'agence existe.
3. Cree le bien via `bienService.creerBien(...)`.
4. Renvoie HTTP `201`.

5. Endpoint **PUT** `/api/biens/:id`

```
router.put('/:id', valider(schemaMettreAJourBien), bienController.updateBien);
```

Role principal: mettre a jour un bien existant.

Fonctionnement:

1. Valide le body avec `schemaMettreAJourBien`.
2. Verifie que le bien existe.
3. Si `agenceId` est present, verifie que l'agence existe.
4. Met a jour via `bienService.mettreAJourBien(id, donnees)`.
5. Renvoie HTTP `200`.

6. Endpoint **DELETE** `/api/biens/:id`

```
router.delete('/:id', bienController.deleteBien);
```

Role principal: supprimer un bien si autorise par les regles metier.

Fonctionnement:

1. Verifie que le bien existe.
2. Verifie qu'il n'a pas de visites planifiees.
3. Supprime via `bienService.supprimerBien(id)`.
4. Renvoie HTTP `200`.
5. Renvoie `409` si le bien est lie a des visites.

7. Endpoint **PATCH** `/api/biens/:id/archiver`

```
router.patch('/:id/archiver', bienController.archiveBien);
```

Role principal: archiver un bien sans le supprimer.

Fonctionnement:

1. Verifie que le bien existe.
 2. Met le `statut` a `ARCHIVE`.
 3. Renvoie HTTP `200`.
-

Documentation des fonctions

1. schemaCreerBien

```
const schemaCreerBien = Joi.object({
  titre: Joi.string().min(3).max(200).required(),
  type: Joi.string().valid(...TYPES_BIEN).required(),
  adresse: Joi.string().min(5).max(255).required(),
  prix: Joi.number().min(0).required(),
  statut: Joi.string().valid(...STATUTS_BIEN).default('DISPONIBLE'),
  agenceId: Joi.number().integer().required(),
});
```

Role principal: valider le body de creation.

Fonctionnement:

1. Controle types et contraintes.
2. Applique `DISPONIBLE` si `statut` absent.
3. Rejette les donnees invalides.

2. schemaMettreAJourBien

```
const schemaMettreAJourBien = Joi.object({
  titre: Joi.string().min(3).max(200),
  type: Joi.string().valid(...TYPES_BIEN),
  adresse: Joi.string().min(5).max(255),
  prix: Joi.number().min(0),
  statut: Joi.string().valid(...STATUTS_BIEN),
  agenceId: Joi.number().integer(),
}).min(1);
```

Role principal: valider le body de mise a jour.

Fonctionnement:

1. Rend tous les champs optionnels.
2. Exige au moins un champ a mettre a jour.
3. Bloque les valeurs invalides.

3. `valider(schema, source = 'body')`

```
const valider = (schema, source = 'body') => {
  return (req, res, next) => {
    const donnees = req[source];
    const { error, value } = schema.validate(donnees, {
      abortEarly: false,
      stripUnknown: true,
    });

    if (error) {
      const details = error.details.map((d) => d.message);
      return next(creerErreur(400, 'Données invalides', details));
    }

    req[source] = value;
    next();
  };
};
```

Role principal: middleware generique de validation Joi.

Fonctionnement:

1. Valide les donnees entrantes.
2. Nettoie les champs non reconnus.
3. Transmet une erreur `400` en cas d'echec.
4. Passe au controller si valide.

4. `getCatalogue()`

```
const getCatalogue = asyncHandler(async (req, res) => {
  const { type, statut, prixMin, prixMax } = req.query;

  const filters = {
    catalogue: true,
    type,
    statut,
    prixMin: prixMin ? parseFloat(prixMin) : undefined,
    prixMax: prixMax ? parseFloat(prixMax) : undefined
  };

  const biens = await bienService.trouverTousBiens(filters);
  sendResponse(res, 200, 'Catalogue récupéré avec succès', biens);
});
```

Role principal: controller du catalogue public.

Fonctionnement:

1. Construit des filtres depuis la query.
2. Exclut les archives via `catalogue: true`.
3. Retourne la liste en `200`.

5. `getBiens()`

```
const getBiens = asyncHandler(async (req, res) => {
  const { type, statut, agenceId, prixMin, prixMax } = req.query;

  const filters = {
    type,
    statut,
    agenceId,
    prixMin: prixMin ? parseFloat(prixMin) : undefined,
    prixMax: prixMax ? parseFloat(prixMax) : undefined
  };

  const biens = await bienService.trouverTousBiens(filters);
  sendResponse(res, 200, 'Biens récupérés avec succès', biens);
});
```


Role principal: controller de lecture liste des biens.

Fonctionnement:

1. Lit les filtres fournis.
2. Appelle le service de recherche.
3. Renvoie `200` avec les biens trouvés.

6. `getBienById()`

```
const getBienById = asyncHandler(async (req, res) => {
  const { id } = req.params;
  const bien = await bienService.trouverBienParId(id);

  if (!bien) {
    throw new HttpError(404, 'Bien non trouvé');
  }

  sendResponse(res, 200, 'Bien récupéré avec succès', bien);
});
```

Role principal: controller de lecture detail par ID.

Fonctionnement:

1. Recupere `id` depuis l'URL.
2. Interroge le service.
3. Retourne `404` si absent, sinon `200`.

7. createBien()

```
const createBien = asyncHandler(async (req, res) => {
  const { titre, type, adresse, prix, statut, agenceId } = req.body;

  const agence = await agenceService.trouverAgenceParId(agenceId);
  if (!agence) {
    throw new HttpError(400, `L'agence avec l'ID ${agenceId} n'existe pas`);
  }

  const bien = await bienService.creerBien({
    titre,
    type,
    adresse,
    prix: parseFloat(prix),
    statut: statut || 'DISPONIBLE',
    agenceId: parseInt(agenceId)
  });

  sendResponse(res, 201, 'Bien créé avec succès', bien);
});
```

Role principal: controller de creation.

Fonctionnement:

1. Lit les donnees du body valide.
2. Verifie l'agence.
3. Convertit les types numeriques.
4. Cree le bien et retourne `201`.

8. `updateBien()`

```
const updateBien = asyncHandler(async (req, res) => {
  const { id } = req.params;
  const { titre, type, adresse, prix, statut, agenceId } = req.body;

  const bienExist = await bienService.trouverBienParId(id);
  if (!bienExist) {
    throw new HttpError(404, 'Bien non trouvé');
  }

  if (agenceId) {
    const agence = await agenceService.trouverAgenceParId(agenceId);
    if (!agence) {
      throw new HttpError(400, `L'agence avec l'ID ${agenceId} n'existe pas`);
    }
  }

  const bien = await bienService.mettreAJourBien(id, {
    titre,
    type,
    adresse,
    prix: prix ? parseFloat(prix) : undefined,
    statut,
    agenceId: agenceId ? parseInt(agenceId) : undefined
  });

  sendResponse(res, 200, 'Bien mis à jour avec succès', bien);
});
```

Role principal: controller de mise a jour.

Fonctionnement:

1. Controle l'existence du bien.
2. Controle optionnellement l'agence.
3. Applique la mise a jour.
4. Retourne `200`.

9. deleteBien()

```
const deleteBien = asyncHandler(async (req, res) => {
  const { id } = req.params;

  const bien = await bienService.trouverBienParId(id);
  if (!bien) {
    throw new HttpError(404, 'Bien non trouvé');
  }

  const aDesVisites = await bienService.bienADesVisites(id);
  if (aDesVisites) {
    throw new HttpError(409, 'Impossible de supprimer : le bien a des visites planifiées');
  }

  await bienService.supprimerBien(id);
  sendResponse(res, 200, 'Bien supprimé avec succès');
});
```

Role principal: controller de suppression.

Fonctionnement:

1. Verifie existence du bien.
2. Verifie absence de visites.
3. Supprime si autorise.
4. Retourne 409 si conflit metier.

10. archiveBien()

```
const archiveBien = asyncHandler(async (req, res) => {
  const { id } = req.params;

  const bien = await bienService.trouverBienParId(id);
  if (!bien) {
    throw new HttpError(404, 'Bien non trouvé');
  }

  const bienArchive = await bienService.archiverBien(id);
  sendResponse(res, 200, 'Bien archivé avec succès', bienArchive);
});
```

Role principal: controller d'archivage.

Fonctionnement:

1. Verifie existence du bien.
2. Passe le statut a `ARCHIVE`.
3. Retourne `200`.

11. `trouverTousBiens(filtres = {})` dans `bien.service`

```
async function trouverTousBiens(filtres = {}) {  
  return bienRepo.trouverTousBiens(filtres);  
}
```

Role principal: deleguer la lecture liste au repository.

Fonctionnement:

1. Recoit des filtres optionnels.
2. Delege la construction de requete au repository.

12. `trouverBienParId(id)` dans `bien.service`

```
async function trouverBienParId(id) {  
  const bien = await bienRepo.trouverBienParId(id);  
  if (!bien) {  
    throw new HttpError(404, 'Bien non trouvé');  
  }  
  return bien;  
}
```

Role principal: regle metier d'existence du bien.

Fonctionnement:

1. Interroge le repository.
2. Leve `404` si absent.
3. Retourne le bien sinon.

13. `creerBien(donnees)` dans `bien.service`

```
async function creerBien(donnees) {  
  const agence = await agenceRepo.trouverAgenceParId(donnees.agenceId);  
  if (!agence) {  
    throw new HttpError(400, `L'agence avec l'ID ${donnees.agenceId} n'existe pas`);  
  }  
  return bienRepo.creer(donnees);  
}
```

Role principal: logique metier de creation.

Fonctionnement:

1. Verifie l'agence.
2. Cree le bien en base.

14. `mettreAJourBien(id, donnees)` dans `bien.service`

```
async function mettreAJourBien(id, donnees) {  
  await trouverBienParId(id);  
  if (donnees.agenceId) {  
    const agence = await agenceRepo.trouverAgenceParId(donnees.agenceId);  
    if (!agence) {  
      throw new HttpError(400, `L'agence avec l'ID ${donnees.agenceId} n'existe pas`);  
    }  
  }  
  return bienRepo.mettreAJour(id, donnees);  
}
```

Role principal: logique metier de mise a jour.

Fonctionnement:

1. Valide existence du bien.
2. Valide agence si modifiee.
3. Met a jour l'enregistrement.

15. `supprimerBien(id)` dans `bien.service`

```
async function supprimerBien(id) {  
  await trouverBienParId(id);  
  const aDesVisites = await bienRepo.bienADesVisites(id);  
  if (aDesVisites) {  
    throw new HttpError(409, 'Impossible de supprimer : le bien a des visites planifiées');  
  }  
  return bienRepo.supprimer(id);  
}
```

Role principal: logique metier de suppression.

Fonctionnement:

1. Valide existence.
2. Bloque suppression si visites associees.
3. Supprime si aucune contrainte.

16. `archiverBien(id)` dans `bien.service`

```
async function archiverBien(id) {  
  await trouverBienParId(id);  
  return bienRepo.archiverBien(id);  
}
```

Role principal: logique metier d'archivage.

Fonctionnement:

1. Verifie existence.
2. Delege la mise a jour du statut au repository.

17. `trouverTousBiens(filtres = {})` dans `bien.repo`

```
async function trouverTousBiens(filtres = {}) {
  const where = {};

  if (filtres.catalogue) {
    where.statut = { not: 'ARCHIVE' };
  }

  if (filtres.statut) where.statut = filtres.statut;
  if (filtres.type) where.type = filtres.type;

  if (filtres.agenceId) {
    where.agenceId = convertirId(filtres.agenceId, "ID de l'agence");
  }

  if (filtres.prixMin || filtres.prixMax) {
    where.prix = {};
    if (filtres.prixMin) where.prix.gte = parseFloat(filtres.prixMin);
    if (filtres.prixMax) where.prix.lte = parseFloat(filtres.prixMax);
  }

  return prisma.bien.findMany({ where });
}
```

Role principal: construire les filtres DB et lire la collection.

Fonctionnement:

1. Construit `where` dynamiquement.
2. Gere filtre catalogue.
3. Gere plage de prix et agence.
4. Execute `findMany` Prisma.

18. `trouverBienParId(id)` dans `bien.repo`

```
async function trouverBienParId(id) {
  return bienBase.trouverParId(id, {
    include: {
      agence: { select: { id: true, code: true, nom: true } },
      visites: {
        include: { client: { select: { id: true, prenom: true, nom: true } } },
        orderBy: { dateVisite: 'desc' },
      },
    },
    _count: { select: { visites: true } },
  },
  {});
}
```

Role principal: lire un bien avec ses relations.

Fonctionnement:

1. Passe par la methode generique `trouverParId`.
2. Inclut agence, visites et compteur.

19. `bienADesVisites(id)` dans `bien.repo`

```
async function bienADesVisites(id) {
  const bienId = convertirId(id, "ID du bien");
  const count = await prisma.visite.count({ where: { bienId } });
  return count > 0;
}
```

Role principal: verifier le lien entre bien et visites.

Fonctionnement:

1. Compte les visites du bien.
2. Retourne `true` si au moins une visite existe.

20. archiverBien(id) dans bien.repo

```
async function archiverBien(id) {
  const idNumber = convertirId(id, "ID du bien");

  return prisma.bien.update({
    where: { id: idNumber },
    data: { statut: 'ARCHIVE' }
  });
}
```

Role principal: mise a jour technique du statut en base.

Fonctionnement:

1. Convertit l'ID.
2. Execute `update` Prisma vers `ARCHIVE`.

21. convertirId(id, nom = 'ID')

```
function convertirId(id, nom = 'ID') {
  if (id === undefined || id === null) {
    throw new Error(`Le ${nom} est requis`);
  }
  if (typeof id === 'number') return id;

  const idNumber = parseInt(id);
  if (isNaN(idNumber)) {
    throw new Error(`Le ${nom} doit être un nombre valide (reçu: ${id})`);
  }

  return idNumber;
}
```

Role principal: securiser les IDs avant requete DB.

Fonctionnement:

1. Refuse ID vide.
2. Convertit string vers entier.
3. Leve une erreur claire si invalide.

22. `sendResponse(res, statusCode, message, data = null)`

```
function sendResponse(res, statusCode, message, data = null) {  
  const response = {  
    success: statusCode >= 200 && statusCode < 300,  
    message,  
    timestamp: new Date().toISOString()  
  };  
  
  if (data !== null) {  
    response.data = data;  
  }  
  
  return res.status(statusCode).json(response);  
}
```

Role principal: standardiser le format des reponses API.

Fonctionnement:

1. Construit une reponse uniforme.
2. Ajoute `data` si disponible.
3. Envoie le status HTTP demande.

23. `asyncHandler(fn)`

```
const asyncHandler = (fn) => {  
  return (req, res, next) => {  
    Promise.resolve(fn(req, res, next)).catch(next);  
  };  
};
```

Role principal: capturer les erreurs async automatiquement.

Fonctionnement:

1. Execute la fonction asynchrone.
2. Redirige les erreurs vers `next(err)`.

24. errorHandler(err, req, res, next)

```
if (err instanceof HttpError) {
  return res.status(err.statusCode).json({
    succes: false,
    message: err.message,
    ...(err.details && { details: err.details }),
  });
}
```

Role principal: convertir les exceptions en reponses HTTP propres.

Fonctionnement:

1. Gere erreurs Prisma (P2002 , P2025 , P2003).
2. Gere erreurs metier HttpError .
3. Retourne 500 pour les erreurs non prevues.

Resume du flux complet du module bien

1. La route choisit le controller selon l'endpoint.
2. Les endpoints POST et PUT passent par la validation Joi.
3. Le controller construit les donnees et applique les verifications immediates.
4. Le service applique les regles metier.
5. Le repository execute les requetes Prisma.
6. sendResponse renvoie une structure JSON standard.
7. errorHandler transforme les erreurs en reponses HTTP coherentes.

Documentation du Module de Gestion des Visites

Vue d'ensemble

Ce document explique tout le module `visite` de l'API de gestion immobiliere. L'objectif est de decrire clairement:

- 1. Les endpoints associes a `visite` .
- 2. Les donnees attendues (body, query, params).
- 3. Les fonctions appelees dans l'ordre.
- 4. Le role de chaque couche (route, validation, controller, service, repository, reponse, erreurs).

Le module `visite` expose les endpoints suivants:

- 1. `GET /api/visites`
- 2. `GET /api/visites/:id`
- 3. `POST /api/visites`
- 4. `PATCH /api/visites/:id/statut`
- 5. `PATCH /api/visites/:id/annuler`
- 6. `PATCH /api/visites/:id/confirmer`
- 7. `PATCH /api/visites/:id/effectuer`
- 8. `DELETE /api/visites/:id`

Architecture en couches:

- 1. Route
 - 2. Validation
 - 3. Controller
 - 4. Services
 - 5. Repositories
 - 6. Reponse standard et gestion d'erreurs
-

Structure des donnees

1) Body de creation (`POST /api/visites`)

```
{
  "clientId": 1,
  "bienId": 1,
  "dateVisite": "2026-12-25T10:00:00Z",
  "commentaire": "Je veux visiter samedi"
}
```

Regles principales:

1. `clientId` : entier obligatoire, client existant.
2. `bienId` : entier obligatoire, bien existant.
3. `dateVisite` : date ISO obligatoire et dans le futur.
4. `commentaire` : optionnel.
5. Pas de doublon (`clientId + bienId + dateVisite`).
6. Le bien doit etre visitable (pas `LOUE` , `VENDU` , `ARCHIVE`).

2) Body de mise a jour statut (`PATCH /api/visites/:id/statut`)

```
{
  "statut": "CONFIRMEE",
  "commentaire": "Visite confirmée"
}
```

Regles principales:

1. `statut` obligatoire dans: `DEMANDEE` , `CONFIRMEE` , `ANNULEE` , `EFFECTUEE` .
2. `commentaire` optionnel.

3) Query params (`GET /api/visites`)

```
clientId, bienId, statut
```

1. `clientId` : filtre par client.
2. `bienId` : filtre par bien.
3. `statut` : filtre par statut.

4) Parametre path (:id)

```
/api/visites/:id
```

1. L'ID est converti avec `convertirId(...)`.
2. Si invalide, erreur `400`.

5) Modele Prisma cible

```
model Visite {
  id          Int          @id @default(autoincrement())
  clientId    Int
  bienId      Int
  dateVisite  DateTime
  statut      StatutVisite @default(DEMANDEE)
  commentaire String?
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt

  client Client @relation(fields: [clientId], references: [id])
  bien  Bien   @relation(fields: [bienId], references: [id])

  @@unique([clientId, bienId, dateVisite])
  @@map("visites")
}
```

Documentation des endpoints

1. Endpoint `GET /api/visites`

```
router.get('/', visiteController.getVisites);
```

Role principal: recuperer la liste des visites avec filtres.

Fonctionnement:

1. Lit `clientId`, `bienId`, `statut` depuis query.
2. Appelle `visiteService.trouverToutesVisites(filtres)`.
3. Renvoie HTTP `200`.

2. Endpoint `GET /api/visites/:id`

```
router.get('/:id', visiteController.getVisiteById);
```

Role principal: recuperer une visite par ID.

Fonctionnement:

1. Lit `id` dans URL.
2. Appelle `visiteService.trouverVisiteParId(id)`.
3. Renvoie `200` si trouve, `404` sinon.

3. Endpoint `POST /api/visites`

```
router.post('/', valider(schemaCreerVisite), visiteController.createVisite);
```

Role principal: planifier une nouvelle visite.

Fonctionnement:

1. Valide le body.
2. Verifie client et bien.
3. Verifie que le bien est visitable.
4. Verifie qu'il n'y a pas de doublon.
5. Cree la visite et renvoie `201`.

4. Endpoint `PATCH /api/visites/:id/statut`

```
router.patch('/:id/statut', valider(schemaMettreAJourVisite), visiteController.updateStatutVisite);
```

Role principal: changer le statut d'une visite.

Fonctionnement:

1. Valide `statut` et `commentaire`.
2. Verifie l'existence de la visite.
3. Met a jour la visite.
4. Renvoie `200`.

5. Endpoint `PATCH /api/visites/:id/annuler`

```
router.patch('/:id/annuler', visiteController.annulerVisite);
```

Role principal: passer le statut a `ANNULEE`.

6. Endpoint `PATCH /api/visites/:id/confirmer`

```
router.patch('/:id/confirmer', visiteController.confirmerVisite);
```

Role principal: passer le statut a `CONFIRMEE`.

7. Endpoint `PATCH /api/visites/:id/effectuer`

```
router.patch('/:id/effectuer', visiteController.effectuerVisite);
```

Role principal: passer le statut a `EFFECTUEE`.

8. Endpoint `DELETE /api/visites/:id`

```
router.delete('/:id', visiteController.deleteVisite);
```

Role principal: supprimer une visite existante.

Fonctionnement:

1. Verifie existence de la visite.
2. Supprime la visite.
3. Renvoie `200`.

Documentation des fonctions

1. `schemaCreerVisite`

```
const schemaCreerVisite = Joi.object({
  clientId: Joi.number().integer().required(),
  bienId: Joi.number().integer().required(),
  dateVisite: Joi.date().iso().greater('now').required(),
  commentaire: Joi.string().max(500).optional().allow(''),
});
```

Role principal: valider la creation d'une visite.

2. `schemaMettreAJourVisite`

```
const schemaMettreAJourVisite = Joi.object({
  statut: Joi.string().valid('DEMANDEE', 'CONFIRMEE', 'ANNULEE', 'EFFECTUEE').required(),
  commentaire: Joi.string().max(500).optional().allow(''),
});
```

Role principal: valider la mise a jour de statut.

3. `valider(schema, source = 'body')`

```
const { error, value } = schema.validate(donnees, {
  abortEarly: false,
  stripUnknown: true,
});
```

Role principal: middleware de validation Joi.

4. `getVisites()`

```
const getVisites = asyncHandler(async (req, res) => {
  const { clientId, bienId, statut } = req.query;
  const filtros = {};
  if (clientId) filtros.clientId = clientId;
  if (bienId) filtros.bienId = bienId;
  if (statut) filtros.statut = statut;

  const visites = await visiteService.trouverToutesVisites(filtros);
  sendResponse(res, 200, 'Visites récupérées avec succès', visites);
});
```

Role principal: controller de lecture liste visites.

5. `getVisiteById()`

```
const getVisiteById = asyncHandler(async (req, res) => {
  const { id } = req.params;
  const visite = await visiteService.trouverVisiteParId(id);
  sendResponse(res, 200, 'Visite récupérée avec succès', visite);
});
```


Role principal: controller de lecture detail visite.

6. createVisite()

```
const createVisite = asyncHandler(async (req, res) => {
  const { clientId, bienId, dateVisite, commentaire } = req.body;

  await clientService.trouverClientParId(clientId);
  const bien = await bienService.trouverBienParId(bienId);

  const estVisitable = await bienService.bienEstVisitable(bienId);
  if (!estVisitable) {
    throw new HttpError(422, `Le bien n'est pas disponible pour une visite (statut: ${bien.statut})`);
  }

  const doublon = await visiteService.verifierDoublonVisite(clientId, bienId, dateVisite);
  if (doublon) {
    throw new HttpError(409, 'Une visite existe déjà pour ce client, ce bien à cette date');
  }

  const visite = await visiteService.planifierVisite({
    clientId: parseInt(clientId),
    bienId: parseInt(bienId),
    dateVisite: new Date(dateVisite),
    commentaire,
    statut: 'DEMANDEE'
  });

  sendResponse(res, 201, 'Visite planifiée avec succès', visite);
});
```

Role principal: controller de planification de visite.

7. `updateStatutVisite()`

```
const updateStatutVisite = asyncHandler(async (req, res) => {  
  const { id } = req.params;  
  const { statut, commentaire } = req.body;  
  
  const visiteExist = await visiteService.trouverVisiteParId(id);  
  const visite = await visiteService.mettreAJourVisite(id, {  
    statut,  
    commentaire: commentaire || visiteExist.commentaire  
  });  
  
  sendResponse(res, 200, 'Statut de la visite mis à jour avec succès', visite);  
});
```

Role principal: controller de mise a jour de statut via body.

8. `annulerVisite()`, `confirmerVisite()`, `effectuerVisite()`

```
const annulerVisite = asyncHandler(async (req, res) => {  
  const visite = await visiteService.annulerVisite(req.params.id);  
  sendResponse(res, 200, 'Visite annulée avec succès', visite);  
});
```

Role principal: controllers raccourcis pour transitions de statut.

Fonctionnement:

1. Recoivent `id`.
2. Appellent la methode service correspondante.
3. Renvoient `200`.

9. `deleteVisite()`

```
const deleteVisite = asyncHandler(async (req, res) => {  
  const { id } = req.params;  
  await visiteService.supprimerVisite(id);  
  sendResponse(res, 200, 'Visite supprimée avec succès');  
});
```

Role principal: controller de suppression visite.

10. trouverToutesVisites(filtres = {}) dans visite.service

```
async function trouverToutesVisites(filtres = {}) {  
  return visiteRepo.trouverToutesVisites(filtres);  
}
```

Role principal: deleguer la lecture liste au repository.

11. trouverVisiteParId(id) dans visite.service

```
async function trouverVisiteParId(id) {  
  const visite = await visiteRepo.trouverVisiteParId(id);  
  if (!visite) throw new HttpError(404, 'Visite non trouvée');  
  return visite;  
}
```

Role principal: regle metier d'existence visite.

12. `planifierVisite(donnees)` dans `visite.service`

```
async function planifierVisite(donnees) {
  const { clientId, bienId, dateVisite, commentaire } = donnees;

  const client = await clientRepo.trouverClientParId(clientId);
  if (!client) throw new HttpError(400, `Le client avec l'ID ${clientId} n'existe pas`);

  const bien = await bienRepo.trouverBienParId(bienId);
  if (!bien) throw new HttpError(400, `Le bien avec l'ID ${bienId} n'existe pas`);

  const estVisitable = await bienRepo.bienEstVisitable(bienId);
  if (!estVisitable) {
    throw new HttpError(422, `Le bien n'est pas disponible pour une visite (statut: ${bien.statut})`);
  }

  const doublon = await visiteRepo.verifierDoublonVisite(clientId, bienId, dateVisite);
  if (doublon) throw new HttpError(409, 'Une visite existe déjà pour ce client, ce bien à cette date');

  return visiteRepo.creer({
    clientId,
    bienId,
    dateVisite: new Date(dateVisite),
    statut: 'DEMANDEE',
    commentaire: commentaire || null,
  });
}
```

Role principal: logique metier centrale de creation visite.

13. `mettreAJourVisite(id, donnees)` dans `visite.service`

```
async function mettreAJourVisite(id, donnees) {
  await trouverVisiteParId(id);
  return visiteRepo.mettreAJour(id, donnees);
}
```

Role principal: logique metier de mise a jour de visite.

14. annulerVisite(id), confirmerVisite(id), effectuerVisite(id) dans visite.service

```
async function annulerVisite(id) {  
  return updateStatutVisite(id, 'ANNULEE');  
}
```

Role principal: changer statut via transitions rapides.

15. supprimerVisite(id) dans visite.service

```
async function supprimerVisite(id) {  
  await trouverVisiteParId(id);  
  return visiteRepo.supprimer(id);  
}
```

Role principal: logique metier de suppression visite.

16. trouverToutesVisites(filtres = {}) dans visite.repo

```
async function trouverToutesVisites(filtres = {}) {  
  const where = {};  
  if (filtres.clientId) where.clientId = convertirId(filtres.clientId, "ID du client");  
  if (filtres.bienId) where.bienId = convertirId(filtres.bienId, "ID du bien");  
  if (filtres.statut) where.statut = filtres.statut;  
  
  return prisma.visite.findMany({ where });  
}
```

Role principal: filtrer et lire les visites.

17. trouverVisiteParId(id) dans visite.repo

```
async function trouverVisiteParId(id) {  
  return visiteBase.trouverParId(id, {  
    include: {  
      client: true,  
      bien: { include: { agence: { select: { code: true, nom: true } } } },  
    },  
  });  
}
```

Role principal: lire une visite avec ses relations.

18. `verifierDoublonVisite(clientId, bienId, dateVisite)` dans `visite.repo`

```
async function verifierDoublonVisite(clientId, bienId, dateVisite) {
  const clientIdNum = convertirId(clientId, "ID du client");
  const bienIdNum = convertirId(bienId, "ID du bien");

  return prisma.visite.findUnique({
    where: {
      clientId_bienId_dateVisite: {
        clientId: clientIdNum,
        bienId: bienIdNum,
        dateVisite: new Date(dateVisite)
      }
    }
  });
}
```

Role principal: eviter doublons de visites.

19. `convertirId(...)`, `sendResponse(...)`, `asyncHandler(...)`, `errorHandler(...)`

Role principal: socle transversal du module.

Fonctionnement:

1. `convertirId` securise les IDs.
2. `sendResponse` uniformise les reponses.
3. `asyncHandler` capture les erreurs async.
4. `errorHandler` transforme les erreurs en HTTP coherent.

Resume du flux complet du module `visite`

1. La route selectionne le controller selon l'endpoint.
2. `POST` et `PATCH /statut` passent par la validation Joi.
3. Le controller orchestre les verifications metier.
4. Le service applique les regles (existence, visitable, doublon).
5. Le repository execute les requetes Prisma.
6. `sendResponse` renvoie un JSON standard.

7. `errorHandler` formate les erreurs en reponses HTTP propres.